HCMC University Of Technology
Faculty of Computer Science & Engineering

Course: System Performance Evaluation
The Queuing Systems-based Performance Evaluation Project

Phuong-Duy Nguyen

October 19, 2026

Goals: This assignment aims to develop a discrete-event simulation application for modeling and evaluating systems involving asynchronous communication and operations. It emphasizes event-driven modeling, asynchronous task coordination, and performance evaluation.

Contents: The assignment covers the following areas:

- Queuing systems, including single-queue models and queuing networks.
- Non-blocking communication and asynchronous task coordination through event-driven mechanisms.
- Discrete-event simulation methods for system evaluation, sensitivity analysis, and optimization.

Students will also map real-world problem parameters, i.e. asynchronous non-blocking communication operation, to queuing-system parameters and design simulation experiments to analyze system behavior. Furthermore, students will leverage simulation tools to verify and validate system performance.

Results: Upon completing this assignment, students should be able to formulate queuing models, implement a discrete-event simulation, evaluate its outcomes, and compose performance reports on asynchronous system designs.

1 Objectives

This project focuses on evaluating the performance of an application modeled as a queuing system. The application may be implemented using a discrete-event simulation. Students will map problem parameters to the corresponding queuing-system parameters.

The performance-evaluation process consists of the following steps:

1. **Define system goals:** Establish the system boundaries and objectives.
2. **Identify services and outcomes:** Specify the services provided and their possible outcomes.
3. **Select metrics:** Define the criteria used to evaluate performance.
4. **List parameters:** Identify the system and workload parameters.
5. **Identify study factors:** Select the parameters to vary during testing.
6. **Choose an evaluation technique:** Select modeling, simulation, or measurement.
7. **Select a workload:** Define representative workloads or benchmarks.
8. **Design experiments:** Plan experiments to evaluate the system.
9. **Analyze data:** Evaluate the collected data and interpret the results.
10. **Present results:** Communicate the findings.

2 Queuing system

A queuing system provides a framework for analyzing systems in which customers, tasks, or requests arrive for service. It supports performance evaluation, bottleneck identification, and analysis of trade-offs among service capacity, waiting time, and resource utilization.

2.1 Single-queue models

Queuing systems are commonly described using Kendall's notation, $A/S/m/B/K/SD$.
Different combinations of parameter values define distinct models. This assignment requires implementing the queuing systems specified below.

- An M/M/1 queue.
- An M/M/1/K queue with finite capacity.
- An M/M/∞ queue.
- An M/M/n/n queue (Erlang loss system).
- An M/M/c multi-server queue.
- An M/M/c/K queue with finite capacity.

2.2 Queuing network

A queuing network comprises multiple single-queue systems connected in series, in parallel, or in a more general topology. Figure 1 illustrates an example of such a network.

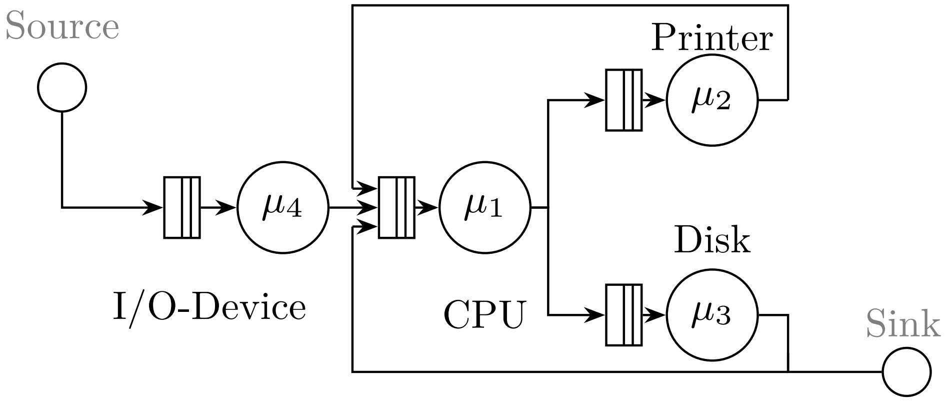
Source
Printer
μ_2
μ_4
μ_1
Disk
μ_3
I/O-Device
CPU
Sink

Figure 1: Example of a multi-queue system.

2.3 Queuing system reference design

As shown in Figure 2, the reference design comprises three components:

1. **Arrival event:** Generates arrivals according to the specified distribution and parameters, such as the mean interarrival time.
2. **Server:** Uses system and process operators to configure the server parameters.
3. **Processing event:** Generates notifications or log entries during event processing.

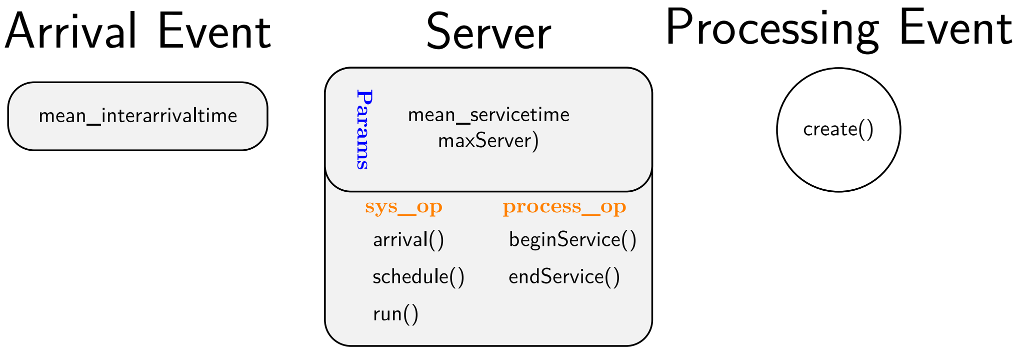
Arrival Event
Server
Processing Event
mean_interarrivaltime
Params
mean_servicetime
maxServer)
sys_op
process_op
arrival()
beginService()
schedule()
endService()
run()
create()

Figure 2: Reference design for the queuing system.

3 Discrete-event simulation

Discrete-event simulation (DES) represents a system as a sequence of state changes triggered by events. It is particularly useful for analyzing complex systems in which activities interact and outcomes depend on timing, resource contention, or variability in decision-making across runs.

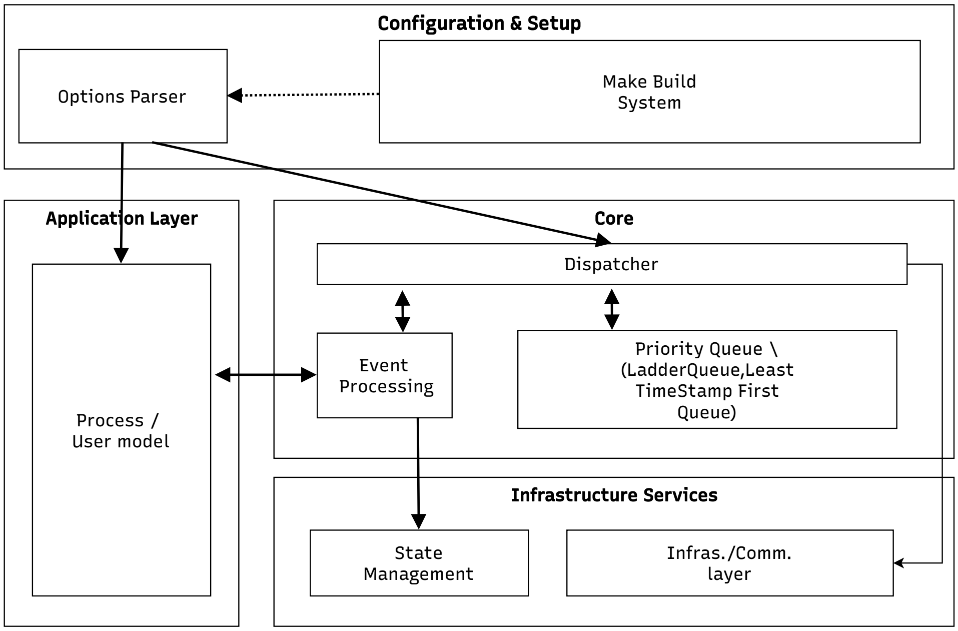
Configuration & Setup
Options Parser
Make Build System
Application Layer
Core
Dispatcher
Process / User model
Event Processing
Priority Queue \ (LadderQueue,Least TimeStamp First Queue)
Infrastructure Services
State Management
Infras./Comm. layer

Figure 3: Discrete-Event Simulation System

Unlike analytical models, DES can evaluate systems without requiring restrictive assumptions, such as the elimination of feedback.
DES supports several forms of analysis:

- **System evaluation:** Uses simulation to estimate performance for specific system configurations and determine whether they meet operational objectives.
- **Sensitivity analysis:** Examines how changes in input parameters affect outcomes, helping identify the factors that most strongly influence performance and where improvements should be prioritized.
- **Optimization and tuning:** Identifies parameter settings that deliver the best performance under specified conditions and supports informed system-design decisions.

The event-driven nature of DES makes it especially well suited for modeling communication and distributed computing systems. As a result, many network simulators are built upon DES principles, representing system behavior through timestamped events and state transitions.
Figure 3 presents the core components of the DES framework.

- **Event:** Represents a timestamped message exchanged between logical processes (LPs). Each event contains its occurrence time, source, destination, and associated payload.
- **Logical process and state:** Each logical process models an independent simulation entity. Its state class stores data describing the entity's current state.
- **Simulation:** Manages the overall execution of the parallel discrete-event simulation (PDES), including initialization, event scheduling, and termination.
- **Dispatcher:** Processes events in timestamp order using a centralized priority queue, which selects the earliest event for execution.

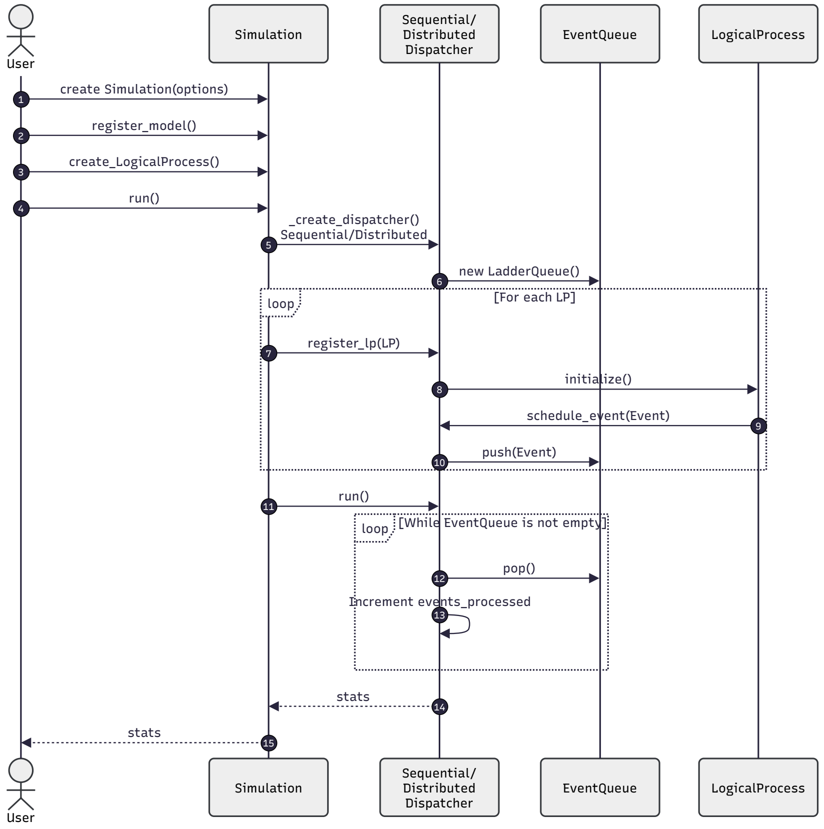
User
Simulation
Sequential/ Distributed Dispatcher
EventQueue
LogicalProcess
1 create Simulation(options)
2 register_model()
3 create_LogicalProcess()
4 run()
5 _create_dispatcher() Sequential/Distributed
6 new LadderQueue()
loop [For each LP]
7 register_lp(LP)
8 initialize()
9 schedule_event(Event)
10 push(Event)
11 run()
loop [While EventQueue is not empty]
12 pop()
13 Increment events_processed
14 stats
15 stats
User
Simulation
Sequential/ Distributed Dispatcher
EventQueue
LogicalProcess

Figure 4: Event simulation.

  - **Distributed event dispatcher:** Uses partitioned LP queues, with a local event queue for each LP, and selects LPs for sequential processing.

- **LadderQueue:** Provides a scalable priority queue for efficiently managing timestamp-ordered events, particularly in simulations involving large event volumes.

Figure 4 illustrates the event-simulation process. Figure 5 shows the event loop process among LogicalProcess, Dispatcher and EventQueue that organizes the repeated processing of events.

4 Non-blocking communication and asynchronous operations

In a non-blocking communication system, communication and computation tasks can progress concurrently without waiting synchronously for communication operations to complete. Messages are buffered when immediate delivery or processing is not feasible. This approach decouples message reception from processing, while queue management must preserve message-arrival order and satisfy synchronization constraints.

The following example demonstrates how Python's `asyncio` library can execute multiple simulation operations concurrently.

Listing 1: Concurrent task execution using Python's asyncio library

```python
import asyncio

```

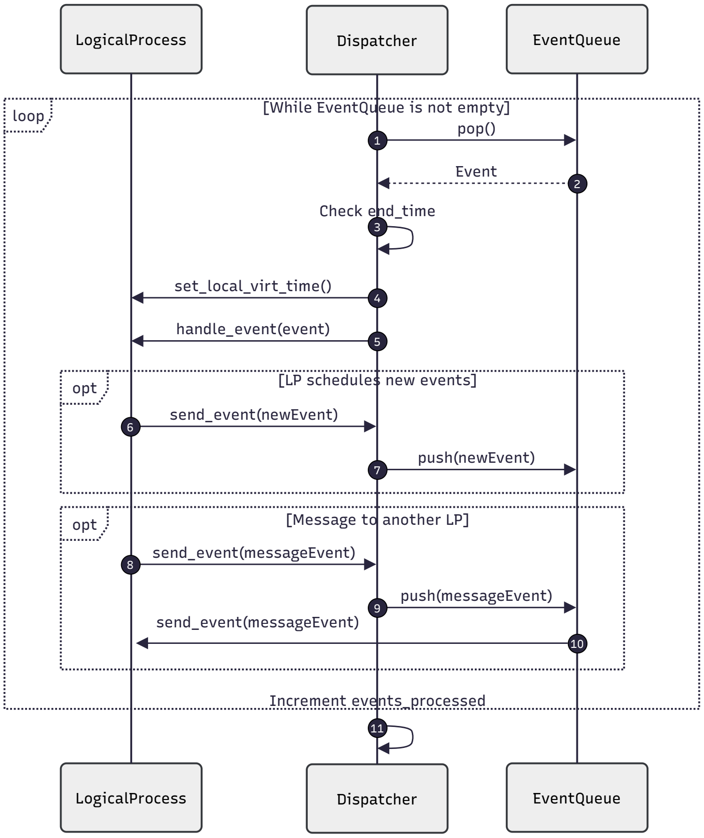
LogicalProcess
Dispatcher
EventQueue
loop
[While EventQueue is not empty]
1
pop()
Event
2
Check end_time
3
set_local_virt_time()
4
handle_event(event)
5
opt
[LP schedules new events]
send_event(newEvent)
6
push(newEvent)
7
opt
[Message to another LP]
send_event(messageEvent)
8
push(messageEvent)
9
send_event(messageEvent)
10
Increment events_processed
11
LogicalProcess
Dispatcher
EventQueue

Figure 5: Event loop

```
async def process_event(event_id):
   await asyncio.sleep(0.1)
   # Simulate asynchronous I/O
   return f"Event {event_id} processed"

async def main():
   results = await asyncio.gather(
    process_event(1),
    process_event(2),
    process_event(3)
    )

    print(results)

if __name__ == "main":
   asyncio.run(main())
```

In this example, each event-processing coroutine represents an independent simulation activity. When a coroutine suspends at an asynchronous wait point, the event loop can schedule another ready coroutine rather than block execution. The `asyncio.gather` function coordinates the operations and returns their results in the order in which the coroutines were supplied, irrespective of their completion order.

In addition, `asyncio.TaskGroup` provides structured task management. It waits for its child tasks to finish when the context exits and offers mechanisms for handling task failures and cancellation. This approach is useful when a simulation phase depends on the completion of several operations.

Alternatively, `asyncio.as_completed` allows results to be consumed as individual tasks finish. This behavior is useful when the simulator must process completed operations immediately rather than wait for the slowest operation.

In a distributed simulation environment, these mechanisms can form a control layer that separates message reception from message processing. A sender initiates a non-blocking send and proceeds with other eligible work, while the receiver handles incoming-message notifications and resumes the corresponding waiting tasks.

Listing 2: Illustrative asynchronous operations in Python

```python
async def operation(...):
   # TODO: Implement the operation body.

# 2a. Define the main coroutine to orchestrate execution:
async def main(...):
   # asyncio.gather runs multiple coroutines concurrently
   # in a default TaskGroup
   results = await asyncio.gather(
       operation(),
       operation(),
   )

# 2b. Define the main coroutine for structured concurrency
async def main():
    async with asyncio.TaskGroup() as tg:
        task1 = tg.create_task(fetch_data(1))
        task2 = tg.create_task(fetch_data(2))

    # Execution pauses here until all tasks in the block have finished.
    print(task1.result(), task2.result())

# 3a. Entry point to run the asynchronous (default) event loop
if __name__ == "__main__":
    asyncio.run(main())

# 3a(bis). Entry point for running the asynchronous singleton and default event loop
if __name__ == "__main__":
    with asyncio.Runner() as runner:
        runner.run(main())

# 3b. Entry point for running the asynchronous custom event loop
if __name__ == "__main__":
    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)


# 3c. Entry point for running the task group using First-Come, First-Served scheduling
if __name__ == "__main__":
   for routine in asyncio.as_completed(
       [main(),
       main(),
       main()]
   ):
       result = await routine
```

The `asyncio.Future` class provides a complementary mechanism for representing a result that may become available later. A coroutine can create a future, associate it with an event identifier, and await its completion. When the corresponding response arrives, the receiving component resolves the future, allowing the suspended coroutine to resume.

Listing 3: Using a future to communicate a result

```
# 1. Producer function (acts as the promise manager)
async def async_worker(future_obj: asyncio.Future):
    future_obj.set_result("Worker output")

# 2. Consumer function (awaits the future)
async def main():
    # Create an empty Future object (the empty container/promise)
    # The event loop must be running; otherwise, an error will occur.
    loop = asyncio.get_running_loop()
    my_future = loop.create_future()

    # Schedule the worker in the background and pass it the future.
    asyncio.create_task(async_worker(my_future))

    # 'await' pauses execution until `my_future.set_result()` is called
    await my_future

if __name__ == "__main__":
    asyncio.run(main())
```

5 Work plan

5.1 Requirements for asynchronous communication and operations in discrete-event simulation

An implementation of asynchronous tasks in a parallel discrete-event simulator requires the following components:

- **Event queue:** Maintains pending simulation events in order of simulation timestamp and scheduling priority.
- **Asynchronous task scheduler:** Schedules and coordinates coroutines without blocking other eligible tasks.
- **Completion notification:** Uses futures or equivalent mechanisms to resume suspended tasks when communication operations complete.
- **Synchronization and causality:** Ensures that events are processed in a causally valid order in accordance with the simulator's synchronization protocol.
- **Task lifecycle management:** Handles task creation, completion, cancellation, timeouts, and communication failures.

Relevant Python mechanisms include `asyncio.gather`, `asyncio.TaskGroup`, `asyncio.Runner`, custom event loops, and `asyncio.Future`.
The implementation must distinguish verification focuses on "Are we building the system right?":

- Verification determines whether a simulation model has been implemented correctly in accordance with its conceptual design and specifications, using techniques such as logical consistency checks, component testing, and computational trace analysis.

whereas validation addresses "Are we building the right system?".

- Validation commonly employs system testing, the model's accuracy, and evaluation in realistic usage scenarios.

simulation time from wall-clock time. Asynchronous waiting, event execution, and timestamp progression must remain under the control of the discrete-event scheduler.

5.2 Project timeline

Phase 1 (one or two weeks): System design

- Define system specifications.

Phase 2 (two weeks): Implementation

- Implement and evaluate the application.

Phase 3 (Week 5): Examinate and finalization

- Conduct cross-team verification and evaluation.

6 Submissions

6.1 Phase 1 submission requirements

- Submit the report to the designated shared folder.
- **File name:** `Spe-Ass-Spec-<GroupName>.pdf`
- **Required contents:**
  - Project specification.
  - Performance metrics.
  - System factors and study parameters.

6.2 Phase 2 submission requirements

- The submitted archive must use the following file name: `Spe-Ass-final-<GroupName>.zip`.
- The archive must include the following:
  - The Phase 1 submission.
  - Architecture and class diagrams.
  - A description of the workload.
  - A description of the experiments.
  - Validation and evaluation.
  - A user manual.

6.3 Phase 3 examination report

- Examine **verification** methods
- Examine the test results: check the evidences that the implementation conforms to the model specifications.
- Examine **validation** data such as reference data, performance metrics, quantitative comparison.

7 Evaluation

7.1 Submission deadline

Submissions will be accepted only if received before the deadline announced on BKeL or in the registration form.

7.2 Scoring

Assignment 1 is worth 10 points:

- Part 1: System design (3 points).
- Part 2: Implementation and experiments (4 points).
  - Maximize the information obtained while minimizing effort.
  - Conduct tests, collect data, and analyze the results.
  - Create clear visualizations to communicate the findings.
- Part 3: Cross-team evaluation (3 points).
  - Assess the validity of the team's design choices (Steps 1–7).
  - Review the team's experimental procedures (Step 8).
  - Assess the team's statistical interpretations and presentation (Steps 9–10).

bkitevsim.core.simulation

index
/Users/pdnguyen/Documents/spe/simulator/bkitevsim/core/simulation.py

`Simulation class.`

Modules

logging

Classes

builtins.object
    Simulation
    SimulationOptions

class **Simulation**(builtins.object)

```
Simulation(options: 'SimulationOptions' = None, models: 'Dict[str, Type[BkSysLogicalProcess]]' = None, factories: 'Dict[str, Callable]' = None)

High-level simulation controller.

Provides a simple interface for creating and running simulations
without directly managing dispatchers and LPs.

Attributes:
    options: Simulation configuration options.
    dispatcher: The active dispatcher instance.
    lps: Dictionary mapping LP IDs to LogicalProcess instances.
```

Methods defined here:

__init__(self, options: 'SimulationOptions' = None, models: 'Dict[str, Type[BkSysLogicalProcess]]' = None, factories: 'Dict[str, Callable]' = None)

```
Initialize a Simulation.

Args:
    options: Simulation configuration options. If None, defaults are used.
    models: Dictionary mapping model names to BkSysLogicalProcess classes.
    factories: Optional dictionary mapping model names to factory functions.
```

create_lp(self, lp_id: 'int', model_name: 'str', **params) -> 'LogicalProcess'

```
Create an LP from a registered model.

Args:
    lp_id: Unique identifier for the LP.
    model_name: Name of the registered model.
    **params: Additional parameters passed to the model constructor.

Returns:
    The created LogicalProcess instance.

Raises:
    ValueError: If model_name is not registered.
```

create_lps(self, model_name: 'str', count: 'int', **params) -> 'List[LogicalProcess]'

```
Create multiple LPs of the same model.

Args:
    model_name: Name of the registered model.
    count: Number of LPs to create.
    **params: Additional parameters passed to each model constructor.

Returns:
    List of created LogicalProcess instances.
```

get_all_lps(self) -> 'List[BkSysLogicalProcess]'

```
Get all LPs.

Returns:
    List of all BkSysLogicalProcess instances.
```

get_lp(self, lp_id: 'int') -> 'Optional[BkSysLogicalProcess]'

```
Get LP by ID.

Args:
    lp_id: LP identifier.

Returns:
    The BkSysLogicalProcess instance or None if not found.
```

get_statistics(self) -> 'Dict[str, Any]'

```
Get combined statistics from all LPs.

Returns:
    Dictionary with dispatcher stats and per-LP statistics.
```

run(self) -> 'Dict[str, Any]'

```
Run the simulation.

Creates dispatcher if needed, registers all LPs, and runs the simulation.

Returns:
    Dictionary with simulation statistics including events_processed and wall_time.
```

Data descriptors defined here:

__dict__

```
dictionary for instance variables (if defined)
```

__weakref__
    list of weak references to the object (if defined)

class **SimulationOptions**(builtins.object)

```
SimulationOptions(end_time: 'float' = 100.0, dispatcher: 'str' = 'sequential', bucket_width: 'float' = 1.0, log_level: 'str' = 'INFO', log_file: 'Optional

Configuration options for a PDES simulation.

Attributes:
    end_time: Simulation end time.
    dispatcher: Dispatcher type ("sequential", "distributed_sequential", or auto for coroutine LPs).
    bucket_width: Bucket width for priority queue.
    log_level: Logging level.
    log_file: Optional log file path.
    stats_file: Optional stats file path.
    verbose: Enable verbose logging.
    model_params: Additional model parameters.
```

Methods defined here:

__init__(self, end_time: 'float' = 100.0, dispatcher: 'str' = 'sequential', bucket_width: 'float' = 1.0, log_level: 'str' = 'INFO', log_file: 'Optional[str]' = None, stats_file: 'Optio

```
Initialize SimulationOptions.

Args:
    end_time: Simulation end time. Defaults to 100.0.
    dispatcher: Dispatcher type. Defaults to "sequential".
    bucket_width: Bucket width for priority queue. Defaults to 1.0.
    log_level: Logging level. Defaults to "INFO".
    log_file: Optional log file path. Defaults to None.
    stats_file: Optional stats file path. Defaults to None.
    verbose: Enable verbose logging. Defaults to False.
    model_params: Additional model parameters. Defaults to None.
```

Data descriptors defined here:

__dict__
    dictionary for instance variables (if defined)

__weakref__
    list of weak references to the object (if defined)

Data

Any = typing.Any
Callable = typing.Callable
Dict = typing.Dict
List = typing.List
Optional = typing.Optional
Type = typing.Type
annotations = _Feature((3, 7, 0, 'beta', 1), (3, 10, 0, 'alpha', 0), 16777216)
logger = <Logger bkitevsim.core.simulation (WARNING)>

bkitevsim.core.event

index
/Users/pdnguyen/Documents/spe/simulator/bkitevsim/core/event.py

Event class for Parallel Discrete Event Simulation.

An Event represents a timestamped message sent between Logical Processes.

Modules

uuid

Classes

builtins.object
    Event

class **Event**(builtins.object)

Event(timestamp: 'float', sender_id: 'int', receiver_id: 'int', event_type: 'str', payload: 'Any' = None, event_id: 'str' = <factory>) -> None

Represents a discrete event in the simulation.

Events are ordered by timestamp for priority queue operations.
The order=True parameter generates comparison methods based on timestamp.

Attributes:
    timestamp: The simulation time when the event occurs.
    sender_id: ID of the LP that sent the event.
    receiver_id: ID of the LP that should receive the event.
    event_type: Type/category of the event.
    payload: Optional data payload associated with the event.
    event_id: Unique identifier for this event.

Methods defined here:

__eq__(self, other)

__ge__(self, other)

__gt__(self, other)

__init__(self, timestamp: 'float', sender_id: 'int', receiver_id: 'int', event_type: 'str', payload: 'Any' = None, event_id: 'str' = <factory>) -> None

__le__(self, other)

__lt__(self, other)

__post_init__(self)
    Validate event after initialization.

    Raises:
        ValueError: If timestamp is negative or LP IDs are negative.

__repr__(self) -> 'str'
    Return repr(self).

Data descriptors defined here:

__dict__
    dictionary for instance variables (if defined)

__weakref__
    list of weak references to the object (if defined)

Data and other attributes defined here:

__annotations__ = {'event_id': 'str', 'event_type': 'str', 'payload': 'Any', 'receiver_id': 'int', 'sender_id': 'int', 'timestamp': 'float'}

__dataclass_fields__ = {'event_id': Field(name='event_id',type='str',default=<datacl...lse,metadata=mappingproxy({}),_field_type=_FIELD), 'event_type': Field(name='event_type',type='str',default=<data...rue,metadata=mappingproxy({}),_field_type=_FIELD), 'payload': Field(name='payload',type='Any',default=None,def...lse,metadata=mappingproxy({}),_field_type=_FIELD), 'receiver_id': Field(name='receiver_id',type='int',default=<dat...rue,metadata=mappingproxy({}),_field_type=_FIELD), 'sender_id': Field(name='sender_id',type='int',default=<datac...rue,metadata=mappingproxy({}),_field_type=_FIELD), 'timestamp': Field(name='timestamp',type='float',default=<dat...rue,metadata=mappingproxy({}),_field_type=_FIELD)}

__dataclass_params__ = _DataclassParams(init=True,repr=True,eq=True,order=True,unsafe_hash=False,frozen=False)

__hash__ = None

payload = None

Functions

create_event(timestamp: 'float', sender_id: 'int', receiver_id: 'int', event_type: 'str', payload: 'Any' = None) -> 'Event'
    Factory function to create a new event.

```
Args:
    timestamp: The simulation time when the event occurs.
    sender_id: ID of the LP that sent the event.
    receiver_id: ID of the LP that should receive the event.
    event_type: Type/category of the event.
    payload: Optional data payload associated with the event.

Returns:
    A new Event instance.
```

Data

Any = typing.Any
Optional = typing.Optional
annotations = _Feature((3, 7, 0, 'beta', 1), (3, 10, 0, 'alpha', 0), 16777216)

bkitevsim.core.logical_process

index
/Users/pdnguyen/Documents/spe/simulator/bkitevsim/core/logical_process.py

```
BkSys Logical Process and State classes for PDES.

A BkSysLogicalProcess represents a simulation entity that processes events.
BkSysLogicalProcessState encapsulates the mutable state of the LP.
```

Modules

copy
typing

Classes

abc.ABC(builtins.object)
    BkSysLogicalProcess
builtins.object
    BkSysLogicalProcessState

class **BkSysLogicalProcess**(abc.ABC)

```
BkSysLogicalProcess(lp_id: 'int', initial_state: 'tp.Optional[BkSysLogicalProcessState]' = None)

Abstract base class for a BkSys Logical Process in PDES (synchronous/event-driven).

Users must implement the required methods to define their simulation behavior.
Each LP has a unique ID and maintains its own Local Virtual Time.

This is the synchronous base class. For coroutine-based LPs, use BkSysCoLogicalProcess.

Attributes:
    lp_id: Unique identifier for this logical process.
    local_virt_time: Current local virtual time of this LP.
    state: Current state of this LP.
```

Method resolution order:
    BkSysLogicalProcess
    abc.ABC
    builtins.object

Methods defined here:

__init__(self, lp_id: 'int', initial_state: 'tp.Optional[BkSysLogicalProcessState]' = None)

```
Initialize a BkSysLogicalProcess.

Args:
    lp_id: Unique identifier for this logical process.
    initial_state: Optional initial state. If not provided, a new BkSysLogicalProcessState is created.
```

__repr__(self) -> 'str'

```
Return repr(self).
```

elapse_virt_time(self, delta: 'float') -> 'float'

```
Elapse the Local Virtual Time by a delta.

This is useful for modeling processing time within handle_event.
The LP's local virtual time is increased by delta.

Args:
    delta: Time delta to elapse (must be non-negative).

Returns:
    New local virtual time.

Raises:
    ValueError: If delta is negative.
```

finalize(self) -> 'None'

```
Called at simulation end for cleanup.
```

get_local_virt_time(self) -> 'float'

```
Get the current Local Virtual Time.
```

```
Returns:
    Current local virtual time.
```

get_output_events(self) -> 'tp.List[Event]'

```
Get and clear the list of events sent since last call.

Returns:
    List of events sent since last call.
```

handle_event(self, event: 'Event') -> 'None'

```
Process an incoming event.

This is the main event handling logic. The LP should:
1. Update its state based on the event
2. Schedule new events by calling send_event()
3. Update local_virt_time to event.timestamp

Args:
    event: The event to process.
```

initialize(self) -> 'None'

```
Called once at simulation start to initialize the LP.

Override to set up initial state, schedule initial events, etc.
```

register_handler(self, event_type: 'str', handler: 'tp.Callable[[Event], None]') -> 'None'

```
Register a handler for a specific event type.

Args:
    event_type: The type of event to handle.
    handler: Callback function to handle the event.
```

send_event(self, event: 'Event') -> 'None'

```
Send an event to another LP.

This method should be called by the LP implementation to schedule
events for other LPs.

Args:
    event: The event to send.
```

set_local_virt_time(self, local_virt_time: 'float') -> 'None'

```
Set the Local Virtual Time.

Args:
    local_virt_time: New local virtual time value.
```

Data descriptors defined here:

__dict__

```
dictionary for instance variables (if defined)
```

__weakref__

```
list of weak references to the object (if defined)
```

Data and other attributes defined here:

ALL_COMPLETED = 'ALL_COMPLETED'

FIRST_COMPLETED = 'FIRST_COMPLETED'

FIRST_EXCEPTION = 'FIRST_EXCEPTION'

__abstractmethods__ = frozenset({'handle_event', 'initialize'})

class **BkSysLogicalProcessState**(builtins.object)

```
BkSysLogicalProcessState(timestamp: 'float' = 0.0, data: 'tp.Dict[str, tp.Any]' = &lt;factory&gt;) -&gt; None

Encapsulates the mutable state of a BkSys Logical Process.

Users should extend this class to define their simulation-specific state.

Attributes:
    timestamp: The virtual time at which this state snapshot was taken.
    data: User-defined state data dictionary.
```

Methods defined here:

__eq__(self, other)

__init__(self, timestamp: 'float' = 0.0, data: 'tp.Dict[str, tp.Any]' = <factory>) -> None

__repr__(self) -> 'str'

```
Return repr(self).
```

clone(self) -> 'BkSysLogicalProcessState'

```
Create a deep copy of this state.

Returns:
    A new BkSysLogicalProcessState instance with deep-copied data.
```

Data descriptors defined here:

__dict__

```
dictionary for instance variables (if defined)
```

__weakref__

```
list of weak references to the object (if defined)
```

Data and other attributes defined here:

__annotations__ = {'data': 'tp.Dict[str, tp.Any]', 'timestamp': 'float'}

__dataclass_fields__ = {'data': Field(name='data',type='tp.Dict[str, tp.Any]',de...rue,metadata=mappingproxy({}),_field_type=_FIELD), 'timestamp': Field(name='timestamp',type='float',default=0.0,...rue,metadata=mappingproxy({}),_field_type=_FIELD)}

__dataclass_params__ = _DataclassParams(init=True,repr=True,eq=True,order=False,unsafe_hash=False,frozen=False)

__hash__ = None

timestamp = 0.0

Data

ALL_COMPLETED = 'ALL_COMPLETED'
FIRST_COMPLETED = 'FIRST_COMPLETED'
FIRST_EXCEPTION = 'FIRST_EXCEPTION'
annotations = _Feature((3, 7, 0, 'beta', 1), (3, 10, 0, 'alpha', 0), 16777216)

bkitevsim.core.logical_coProcess

index
/Users/pdnguyen/Documents/spe/simulator/bkitevsim/core/logical_coProcess.py

```
BkSys Co-Simulation Logical Process and Event Loop for PDES.

This module provides the BkSysCoLogicalProcess base class and BkSysCoEventLoop
for coroutine-based simulation using cooperative multitasking.
```

Modules

asyncio
logging
time
typing

Classes

_asyncio.Future(builtins.object)
    BkSysFuture
bkitevsim.core.logical_process.BkSysLogicalProcess(abc.ABC)
    BkSysCoLogicalProcess
builtins.BaseException(builtins.object)
    asyncio.exceptions.CancelledError
builtins.Exception(builtins.BaseException)
    asyncio.exceptions.TimeoutError
builtins.object
    BkSysCoEventLoop
enum.Enum(builtins.object)
    BkSysCoLPStatus

class **BkSysCoEventLoop**(builtins.object)

```
BkSysCoEventLoop(end_time: 'float' = inf, bucket_width: 'float' = 1.0)

Single-threaded coroutine-based event loop for PDES.

This event loop runs LogicalProcess coroutines cooperatively.
Each LP is a coroutine that yields control via await statements.
The loop manages a priority queue of events and runs ready coroutines.
Uses LadderQueue internally for parallel-capable priority queue operations.

Attributes:
    end_time: Simulation end time.
    current_time: Current simulation time.
    stats: Dictionary containing simulation statistics.
```

Methods defined here:

__init__(self, end_time: 'float' = inf, bucket_width: 'float' = 1.0)

```
Initialize the BkSysCoEventLoop.

Args:
    end_time: Simulation end time. Defaults to infinity.
    bucket_width: Bucket width for priority queue. Defaults to 1.0.
```

async **advance_time**(self, lp_id: 'int', delta: 'float') -> 'float'

```
Advance virtual time by delta and yield control.

Args:
    lp_id: ID of the LP advancing time.
    delta: Time delta to advance.

Returns:
    Elapsed new virtual time.
```

create_future(self)

```
Create a new PDES future.

Returns:
    A new BkSysFuture instance.
```

get_stats(self) -> 'tp.Dict'

```
Get simulation statistics.

Returns:
    Dictionary with simulation statistics.
```

register_lp(self, lp: 'BkSysLogicalProcess') -> 'None'

```
Register a LogicalProcess coroutine.

Args:
        lp: The LogicalProcess to register.

    Raises:
        ValueError: If registered LP is duplicate.
```

run(self, max_iterations: 'int' = None) -> 'tp.Dict'

```
    Run the coroutine event loop.

    Args:
        max_iterations: Optional maximum number of iterations.

    Returns:
        Dictionary with simulation statistics.
```

schedule_event(self, event: 'Event') -> 'None'

```
    Schedule an event for delivery at its timestamp.

    Args:
        event: The event to schedule.
```

schedule_event_at(self, timestamp: 'float', event: 'Event') -> 'None'

```
    Schedule an event at a specific timestamp (for LP use).

    Args:
        timestamp: The timestamp for the event.
        event: The event to schedule.
```

stop(self) -> 'None'

```
    Stop the event loop.

    Args: None
```

async **wait_for_event**(self, lp_id: 'int', event_type: 'str', timeout: 'float' = None) -> 'Event'

```
    Coroutine wait for a specific event type.
    Elaspse time control until an event of the given type arrives.

    Args:
        lp_id: ID of the LP waiting for the event.
        event_type: Type of event to wait for.
        timeout: Optional timeout in seconds.

    Returns:
        The received event.

    Raises:
        TIMEOUT_ERROR: If timeout expires before event arrives.
```

async **wait_until_time**(self, lp_id: 'int', target_time: 'float') -> 'float'

```
    Coroutine wait until virtual time reaches target_time.
    Elaspse time control until the simulation time advances.

    Args:
        lp_id: ID of the LP waiting for time.
        target_time: Target virtual time to wait for.

    Returns:
        Current virtual time (>= target_time).
```

Readonly properties defined here:

current_time

```
    Get current simulation time.

    Returns:
        Current simulation time.
```

Data descriptors defined here:

__dict__

```
    dictionary for instance variables (if defined)
```

__weakref__

```
    list of weak references to the object (if defined)
```

class **BkSysCoLPStatus**(enum.Enum)

```
BkSysCoLPStatus(value, names=None, *, module=None, qualname=None, type=None, start=1)

Status of a coroutine-based LP.

Attributes:
    READY: LP is ready to run.
    WAITING_EVENT: LP is waiting for an event.
    WAITING_TIME: LP is waiting for virtual time to advance.
    SUSPENDED: LP is suspended.
    TERMINATED: LP has terminated.
```

Method resolution order:
- BkSysCoLPStatus
- enum.Enum
- builtins.object

Data and other attributes defined here:

READY = <BkSysCoLPStatus.READY: 'ready'>

SUSPENDED = <BkSysCoLPStatus.SUSPENDED: 'suspended'>

TERMINATED = <BkSysCoLPStatus.TERMINATED: 'terminated'>

WAITING_EVENT = <BkSysCoLPStatus.WAITING_EVENT: 'waiting_event'>

WAITING_TIME = <BkSysCoLPStatus.WAITING_TIME: 'waiting_time'>

Data descriptors inherited from enum.Enum:

name

```
The name of the Enum member.
```

value

```
The value of the Enum member.
```

Readonly properties inherited from enum.EnumMeta:

__members__

```
Returns a mapping of member name->value.

This mapping lists all enum members, including aliases. Note that this
is a read-only view of the internal mapping.
```

class **BkSysCoLogicalProcess**(bkitevsim.core.logical_process.BkSysLogicalProcess)

```
BkSysCoLogicalProcess(lp_id: 'int', initial_state: 'tp.Optional[BkSysLogicalProcessState]' = None, **kwargs)

Base class for Coroutine BkSysLogicalProcesses.
Events are delivered via handle_event().
The default handle_event implementation buffers events.

Attributes:
    _event_loop: Reference to the associated BkSysCoEventLoop.
    _lookahead: Lookahead time for event scheduling.
```

Method resolution order:
- BkSysCoLogicalProcess
- bkitevsim.core.logical_process.BkSysLogicalProcess
- abc.ABC
- builtins.object

Methods defined here:

__init__(self, lp_id: 'int', initial_state: 'tp.Optional[BkSysLogicalProcessState]' = None, **kwargs)

```
Initialize a BkSysCoLogicalProcess.

Args:
    lp_id: Unique identifier for this logical process.
    initial_state: Optional initial state.
    **kwargs: Additional keyword arguments. Supports 'lookahead'.
```

async **advance_time**(self, delta: 'float') -> 'float'

```
Advance virtual time by delta.

Args:
    delta: Time delta to advance.

Returns:
    New current virtual time.

Raises:
    RuntimeError: If event loop is not set.
```

coroutine_run(self) -> 'tp.Awaitable[None]'

```
Main coroutine - must be implemented by subclasses.
```

elapse_virt_time(self, delta: 'float') -> 'float'

```
Elapse the Local Virtual Time by a delta (local to this LP).

Args:
    delta: Time delta to elapse (must be non-negative).

Returns:
    New local virtual time.

Raises:
    ValueError: If delta is negative.
```

handle_event(self, event: 'Event') -> 'None'

```
Handle an incoming event by buffering it for wait_for consumption.
The event is buffered and will be consumed by wait_for/wait_for_multiple.

Args:
    event: The event to handle.
```

initialize(self) -> 'None'

```
Initialize is handled by coroutine_run.
```

send_event(self, event: 'Event') -> 'None'

```
Send an event through the event loop.

Args:
    event: The event to send.
```

set_event_loop(self, event_loop: 'BkSysCoEventLoop') -> 'None'

```
Set the event loop reference (called by dispatcher).

Args:
    event_loop: The BkSysCoEventLoop to associate with this LP.
```

async **wait_for**(self, event_type: 'str', timeout: 'float' = None) -> 'Event'

```
Wait for an event of the given type and wrap the event loop's wait_for_event.

Args:
    event_type: Type of event to wait for.
    timeout: Optional timeout in wall-clock seconds.

Returns:
    The received event.

Raises:
    TIMEOUT_ERROR: If timeout expires before event arrives.
    RuntimeError: If event loop is not set.
```

async **wait_for_multiple**(self, event_types: 'tp.List[str]', timeout: 'float' = None) -> 'Event'

```
Wait for any of multiple event types.

Args:
    event_types: List of event types to wait for.
    timeout: Optional timeout in wall-clock seconds.

Returns:
    The first received event.

Raises:
    TIMEOUT_ERROR: If timeout expires before any event arrives.
    RuntimeError: If event loop is not set.
```

async **wait_until**(self, target_time: 'float') -> 'float'

```
Wait until virtual time reaches target_time.

Args:
    target_time: Target virtual time to wait for.

Returns:
    Current virtual time (which will be >= target_time).

Raises:
    RuntimeError: If event loop is not set.
```

Data and other attributes defined here:

__abstractmethods__ = frozenset()

Methods inherited from bkitevsim.core.logical_process.BkSysLogicalProcess:

__repr__(self) -> 'str'

```
Return repr(self).
```

finalize(self) -> 'None'
Called at simulation end for cleanup.

get_local_virt_time(self) -> 'float'
Get the current Local Virtual Time.

Returns:
    Current local virtual time.

get_output_events(self) -> 'tp.List[Event]'
Get and clear the list of events sent since last call.

Returns:
    List of events sent since last call.

register_handler(self, event_type: 'str', handler: 'tp.Callable[[Event], None]') -> 'None'
Register a handler for a specific event type.

Args:
    event_type: The type of event to handle.
    handler: Callback function to handle the event.

set_local_virt_time(self, local_virt_time: 'float') -> 'None'
Set the Local Virtual Time.

Args:
    local_virt_time: New local virtual time value.

Data descriptors inherited from bkitevsim.core.logical_process.BkSysLogicalProcess:

__dict__
dictionary for instance variables (if defined)

__weakref__
list of weak references to the object (if defined)

Data and other attributes inherited from bkitevsim.core.logical_process.BkSysLogicalProcess:

ALL_COMPLETED = 'ALL_COMPLETED'

FIRST_COMPLETED = 'FIRST_COMPLETED'

FIRST_EXCEPTION = 'FIRST_EXCEPTION'

class **BkSysFuture**(_asyncio.Future)

BkSysFuture(*, loop=None)

BkSys-specific future implementation using asyncio.Future.

Method resolution order:
- BkSysFuture
- _asyncio.Future
- builtins.object

Methods inherited from _asyncio.Future:

__await__(self, /)
Return an iterator to be used in await expression.

__del__(...)

__init__(self, /, *args, **kwargs)
Initialize self. See help(type(self)) for accurate signature.

__iter__(self, /)
Implement iter(self).

__repr__(self, /)
Return repr(self).

add_done_callback(...)
Add a callback to be run when the future becomes done.

The callback is called with a single argument - the future object. If
the future is already done when this is called, the callback is
scheduled with call_soon.

cancel(self, /, msg=None)

Cancel the future and schedule callbacks.

If the future is already done or cancelled, return False. Otherwise, change the future's state to cancelled, schedule the callbacks and return True.

cancelled(self, /)
Return True if the future was cancelled.

done(self, /)
Return True if the future is done.

Done means either that a result / exception are available, or that the future was cancelled.

exception(self, /)
Return the exception that was set on this future.

The exception (or None if no exception was set) is returned only if the future is done. If the future has been cancelled, raises CancelledError. If the future isn't done yet, raises InvalidStateError.

get_loop(self, /)
Return the event loop the Future is bound to.

remove_done_callback(self, fn, /)
Remove all instances of a callback from the "call when done" list.

Returns the number of callbacks removed.

result(self, /)
Return the result this future represents.

If the future has been cancelled, raises CancelledError. If the future's result isn't yet available, raises InvalidStateError. If the future is done and has an exception set, this exception is raised.

set_exception(self, exception, /)
Mark the future done and set an exception.

If the future is already done when this method is called, raises InvalidStateError.

set_result(self, result, /)
Mark the future done and set its result.

If the future is already done when this method is called, raises InvalidStateError.

Class methods inherited from _asyncio.Future:

__class_getitem__(...) from builtins.type

Static methods inherited from _asyncio.Future:

__new__(*args, **kwargs) from builtins.type
Create and return a new object. See help(type) for accurate signature.

CANCELLED_ERROR = class CancelledError(builtins.BaseException)
The Future or Task was cancelled.

Method resolution order:
CancelledError
builtins.BaseException
builtins.object

Data descriptors defined here:

__weakref__
list of weak references to the object (if defined)

Methods inherited from builtins.BaseException:

__delattr__(self, name, /)
Implement delattr(self, name).

__getattribute__(self, name, /)

```
Return getattr(self, name).
```

__init__(self, /, *args, **kwargs)

```
Initialize self.  See help(type(self)) for accurate signature.
```

__reduce__(...)

```
Helper for pickle.
```

__repr__(self, /)

```
Return repr(self).
```

__setattr__(self, name, value, /)

```
Implement setattr(self, name, value).
```

__setstate__(...)

__str__(self, /)

```
Return str(self).
```

with_traceback(...)

```
Exception.with_traceback(tb) --
set self.__traceback__ to tb and return self.
```

Static methods inherited from builtins.BaseException:

__new__(*args, **kwargs) from builtins.type

```
Create and return a new object.  See help(type) for accurate signature.
```

Data descriptors inherited from builtins.BaseException:

__cause__

```
exception cause
```

__context__

```
exception context
```

__dict__

__suppress_context__

__traceback__

args

TIMEOUT_ERROR = class TimeoutError(builtins.Exception)

```
The operation exceeded the given deadline.
```

Method resolution order:

- TimeoutError
- builtins.Exception
- builtins.BaseException
- builtins.object

Data descriptors defined here:

__weakref__

```
list of weak references to the object (if defined)
```

Methods inherited from builtins.Exception:

__init__(self, /, *args, **kwargs)

```
Initialize self.  See help(type(self)) for accurate signature.
```

Static methods inherited from builtins.Exception:

__new__(*args, **kwargs) from builtins.type

```
Create and return a new object.  See help(type) for accurate signature.
```

Methods inherited from builtins.BaseException:

__delattr__(self, name, /)

```
Implement delattr(self, name).
```

__getattribute__(self, name, /)

```
Return getattr(self, name).
```

__reduce__(...)

```
Helper for pickle.
```

__repr__(self, /)

```
Return repr(self).
```

__setattr__(self, name, value, /)

```
Implement setattr(self, name, value).
```

__setstate__(...)

__str__(self, /)

```
Return str(self).
```

with_traceback(...)

```
Exception.with_traceback(tb) --
set self.__traceback__ to tb and return self.
```

Data descriptors inherited from builtins.BaseException:

__cause__

```
exception cause
```

__context__

```
exception context
```

__dict__

__suppress_context__

__traceback__

args

Data

ALL_COMPLETED = 'ALL_COMPLETED'
FIRST_COMPLETED = 'FIRST_COMPLETED'
FIRST_EXCEPTION = 'FIRST_EXCEPTION'
__all__ = ['BkSysCoLPStatus', 'BkSysCoEventLoop', 'BkSysCoLogicalProcess', 'BkSysFuture', 'CANCELLED_ERROR', 'TIMEOUT_ERROR', 'FIRST_COMPLETED', 'FIRST_EXCEPTION', 'ALL_COMPLETED']

bkitevsim.dispatchers.sequential_eventdispatcher /Users/pdnguyen/Documents/spe/simulator/bkitevsim/dispatchers/sequential_eventdispa

```
Sequential Event Dispatcher.

A single-threaded event dispatcher that processes events in strict timestamp order
using a centralized priority queue. Ideal for debugging and small simulations.
```

Modules

logging	time

Classes

builtins.object
    SequentialEventDispatcher
        DistributedEventDispatcher

class **DistributedEventDispatcher**(SequentialEventDispatcher)

```
DistributedEventDispatcher(end_time: 'float' = inf, bucket_width: 'float' = 1.0)

Sequential dispatcher with per-LP queues (partitioned).

Each LP has its own local event queue. The dispatcher selects the LP
with the minimum next event timestamp. This mimics the structure of
parallel dispatchers but runs sequentially.

Attributes:
    _lp_queues: Dictionary mapping LP IDs to their local LadderQueue instances.
    _lp_next_event: Dictionary mapping LP IDs to their next pending event.
```

Method resolution order:
    DistributedEventDispatcher
    SequentialEventDispatcher
    builtins.object

Methods defined here:

__init__(self, end_time: 'float' = inf, bucket_width: 'float' = 1.0)

```
Initialize the DistributedEventDispatcher.

Args:
    end_time: Simulation end time. Defaults to infinity.
    bucket_width: Bucket width for LadderQueue. Defaults to 1.0.
```

register_lp(self, lp: 'LogicalProcess') -> 'None'

```
Register a Logical Process with the dispatcher.

Args:
    lp: The LogicalProcess to register.
```

run(self) -> 'Dict'

```
Run partitioned sequential simulation.

Returns:
    Dictionary with simulation statistics including events_processed and wall_time.
```

schedule_event(self, event: 'Event') -> 'None'

```
Schedule event in the receiver's local queue.

Args:
    event: The event to schedule.
```

Methods inherited from SequentialEventDispatcher:

__repr__(self) -> 'str'

```
Return repr(self).
```

get_all_lps(self) -> 'List[LogicalProcess]'

```
Get all registered LPs.

Returns:
    List of all registered LogicalProcess instances.
```

get_lp(self, lp_id: 'int') -> 'Optional[LogicalProcess]'

```
Get LP by ID.

Args:
    lp_id: LP identifier.

Returns:
    The LogicalProcess instance or None if not found.
```

schedule_initial_events(self) -> 'None'

```
Allow all LPs to schedule their initial events by calling their initialize() method.
```

Data descriptors inherited from SequentialEventDispatcher:

__dict__

```
dictionary for instance variables (if defined)
```

__weakref__

```
list of weak references to the object (if defined)
```

class **SequentialEventDispatcher**(builtins.object)

```
SequentialEventDispatcher(end_time: 'float' = inf, bucket_width: 'float' = 1.0)

Sequential (single-threaded) event dispatcher.

Processes events in strict timestamp order using a central priority queue.
No parallelism - simple and deterministic execution.

Attributes:
    lps: Dictionary mapping LP ID to LogicalProcess instance.
    event_queue: Central priority queue for all events (LadderQueue).
    current_time: Global simulation time.
    end_time: Simulation end time.
    stats: Performance statistics dictionary.
```

Methods defined here:

__init__(self, end_time: 'float' = inf, bucket_width: 'float' = 1.0)

```
Initialize the SequentialEventDispatcher.

Args:
    end_time: Simulation end time. Defaults to infinity.
    bucket_width: Bucket width for LadderQueue. Defaults to 1.0.
```

__repr__(self) -> 'str'

```
Return repr(self).
```

get_all_lps(self) -> 'List[LogicalProcess]'

```
Get all registered LPs.

Returns:
    List of all registered LogicalProcess instances.
```

get_lp(self, lp_id: 'int') -> 'Optional[LogicalProcess]'

```
Get LP by ID.

Args:
    lp_id: LP identifier.

Returns:
    The LogicalProcess instance or None if not found.
```

register_lp(self, lp: 'LogicalProcess') -> 'None'

```
Register a Logical Process with the dispatcher.

Args:
    lp: The LogicalProcess to register.

Raises:
    ValueError: If an LP with the same ID is already registered.
```

run(self) -> 'Dict'

```
Run the simulation until end_time or no more events.

Returns:
    Dictionary with simulation statistics including events_processed and wall_time.
```

schedule_event(self, event: 'Event') -> 'None'

```
Schedule an event for processing.

Args:
    event: The event to schedule.
```

schedule_initial_events(self) -> 'None'

```
Allow all LPs to schedule their initial events by calling their initialize() method.
```

Data descriptors defined here:

__dict__

```
dictionary for instance variables (if defined)
```

__weakref__

```
list of weak references to the object (if defined)
```

Data

Dict = typing.Dict
List = typing.List
Optional = typing.Optional
Type = typing.Type
annotations = _Feature((3, 7, 0, 'beta', 1), (3, 10, 0, 'alpha', 0), 16777216)
logger = <Logger bkitevsim.dispatchers.sequential_eventdispatcher (WARNING)>